

Breathe Karachi: End-to-End AQI Forecasting Pipeline

Project Report — Pearls AQI Predictor

Abdullah Khan Sherwani — June 2026

Abstract

Breathe Karachi is a fully serverless, end-to-end machine learning pipeline that forecasts Karachi's US Air Quality Index (AQI) four days ahead. The system collects raw weather and pollutant data hourly from public APIs, engineers 141 features per day, retrains three ML models daily, and serves predictions through a Streamlit dashboard—all without a persistent server. Three model families are trained and evaluated — Ridge Regression, LightGBM, and LSTM — with the best-performing model each run selected via rolling-origin cross-validation. The live dashboard is available at <https://breathe-karachi.onrender.com>.

1 System Architecture

The pipeline is split into four loosely coupled modules, each a standalone Python script, with MongoDB Atlas as the sole shared state store.

Table 1: Technology stack

Layer	Technology
Data sources	Open-Meteo Archive, Forecast, and Air Quality APIs
Feature & model store	MongoDB Atlas (serverless)
Models	Ridge Regression, LightGBM, LSTM (TensorFlow/Keras)
Automation	GitHub Actions + cron-job.org
Explainability	SHAP
Dashboard	Streamlit + Flask (deployed on Render)

Data flows as follows:

APIs $\xrightarrow{\text{hourly}}$ feature_store $\xrightarrow{\text{engineering}}$ feature_store (processed) $\xrightarrow{\text{daily}}$ model_registry $\rightarrow$ prediction

All pipeline stages authenticate to MongoDB via GitHub Secrets; no file-system state is shared between runs.

2 Feature Pipeline

2.1 Data Acquisition

Historical data is fetched from two Open-Meteo endpoints: the ERA5 archive for weather and the air quality archive for pollutants. The weather backfill extends to 2018-01-01 (~3077 rows stored), but air quality values are only available from August 2022 onward; rows before that date have no AQI labels. Incremental hourly updates append only the latest day.

2.2 Feature Engineering

`preprocess_daily_data.py` applies a fixed 17-step pipeline per run, producing **141 features** per daily row:

- IQR capping on 12 outlier-prone columns; $\log_{1p}$ transforms for $\text{PM}_{2.5}$ and CO
- Wind direction encoded as sine/cosine; stagnant-air binary flag
- Calendar features: month, season dummies, weekday dummies, cyclical sin/cos encodings
- Lag features: AQI at $t-1, \dots, t-14$; $\text{PM}_{2.5}$, CO, NO_2 lags
- Rolling statistics: 3-, 7-, 14-day means, std, min, max; exponential weighted means
- Physics-inspired features: dew point, vapour pressure deficit (VPD), boundary layer height

- Interaction terms: $\text{PM}_{2.5} \times \text{Humidity}$, $\text{CO} \times \text{Temperature}$
- Weather lead features $t+1$ to $t+4$, filled from the Open-Meteo 5-day forecast for the latest rows
- Open-Meteo air-quality leads: AOD, dust, UV index $t+1$ to $t+4$ ($\text{PM}_{2.5}$ leads are engineered but excluded from the production model; see Section 5.3)

The final four rows always have NaN lead features (no future observations exist). These are filled from the Open-Meteo forecast API before upserting. Target columns (AQI_{t+1} through AQI_{t+4}) remain NaN intentionally for those rows.

3 Exploratory Data Analysis

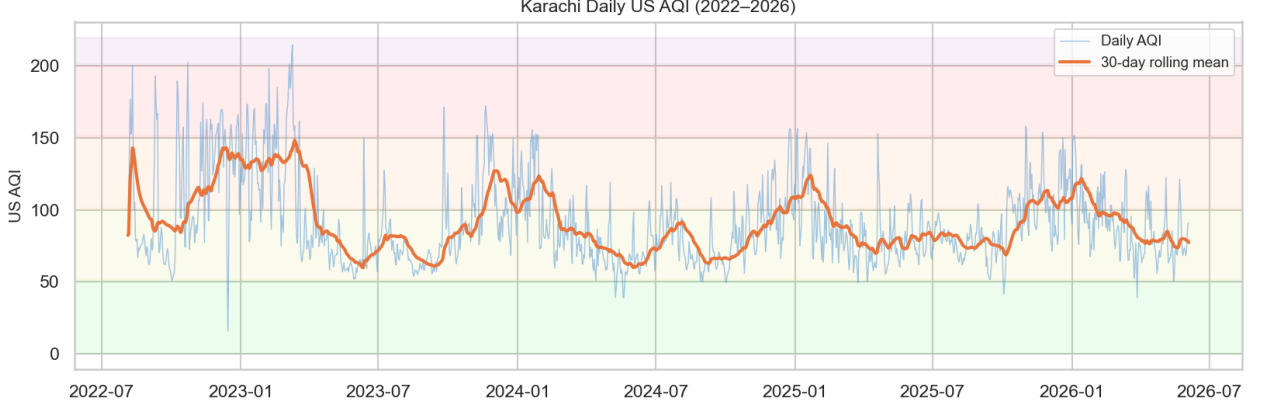
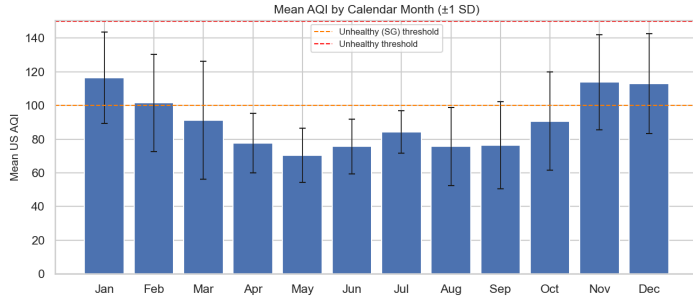
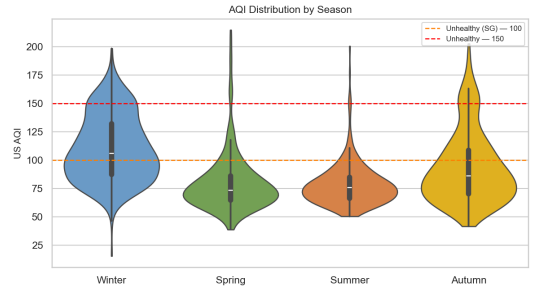


Figure 1: Karachi daily US AQI (Aug 2022–Jun 2026) with 30-day rolling mean. Coloured bands mark EPA categories: Good (green), Moderate (yellow), Unhealthy for Sensitive Groups (orange), Unhealthy (red). Air-quality data is unavailable before Aug 2022; weather backfill extends to 2018. Training uses rows from 2023-01-01 onward.

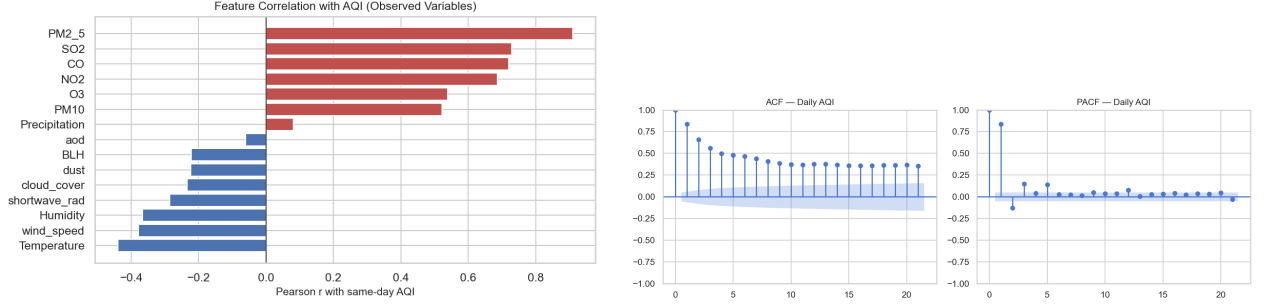


(a) Mean AQI ± 1 SD by calendar month. January, November, and December average above 100 (Unhealthy for Sensitive Groups), driven by temperature inversions that trap surface pollutants. May is the cleanest month (≈ 70 mean AQI).



(b) AQI distribution by meteorological season. Winter has both the highest median and the broadest spread, with a substantial mass above the Unhealthy threshold (red dash). Summer and Spring concentrate below 100.

Figure 2: Strong seasonal structure motivates month and season as engineered features.



(a) Pearson r of observed variables with same-day AQI. $\text{PM}_{2.5}$ ($r = 0.91$) dominates, followed by SO_2 , CO, and NO_2 . Temperature ($r = -0.45$) and wind speed ($r = -0.41$) are the strongest negative predictors — higher wind disperses pollutants.

(b) ACF and PACF of the daily AQI series. ACF decays slowly (ACF at lag 1: 0.84, lag 3: 0.56, lag 4: 0.50), confirming persistent multi-day memory and motivating lag and rolling-window features. PACF cuts off sharply after lag 1, indicating most of the higher-order autocorrelation is mediated by the first lag.

Figure 3: Feature motivation: correlations and autocorrelation structure of the AQI series.

SHAP feature importance. All three models are explained via model-class-appropriate SHAP methods: TreeExplainer for LightGBM, LinearExplainer for Ridge, and Expected Gradients (GradientSHAP via `tf.GradientTape`) for LSTM. Values are persisted to MongoDB after each training run and rendered on the dashboard.

LightGBM (current run): $\text{PM}_{2.5}$ and its log transform dominate across all horizons, followed by the seasonal cycle (`doy_cos`), aerosol lead features (`aod_t1-t3`, `dust_roll_mean_7`), and 14-day AQI rolling statistics.

Ridge: the seasonal cycle (`doy_cos`) ranks first, followed by 7–30-day AQI/ $\text{PM}_{2.5}$ rolling statistics (`PM2.5_ewm_7`, `AQI_roll_min_7`). Consistent with its linear structure, Ridge relies on smoothed pollution history rather than the most recent reading.

LSTM: Expected-Gradients attributions highlight aerosol lead features (`aod_t4`, `aod_t3`) ahead of $\text{PM}_{2.5}$ directly, suggesting the recurrent architecture learns to exploit near-future aerosol optical depth as a forward signal.

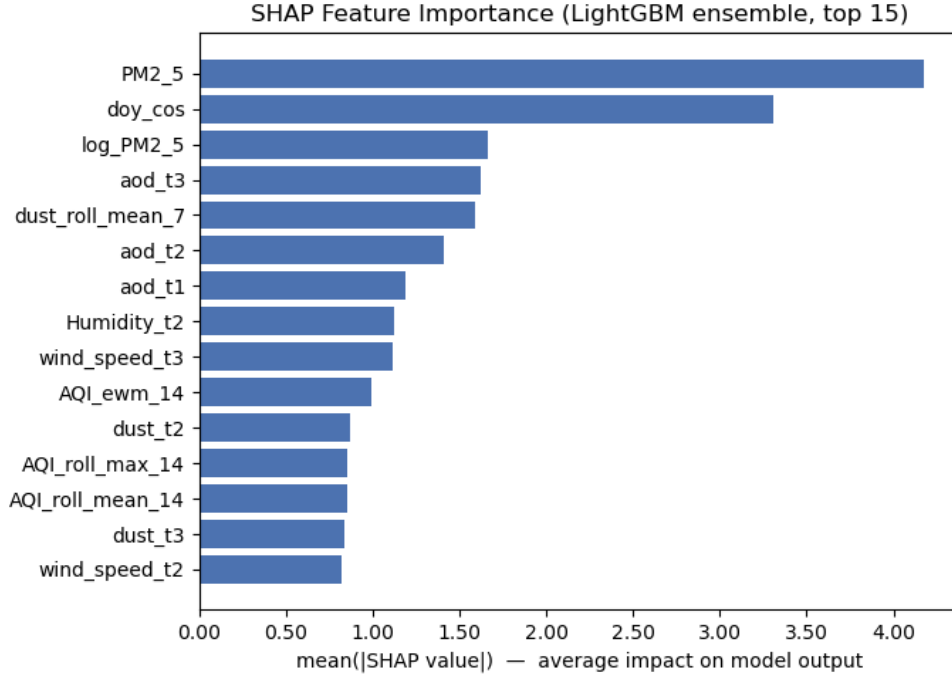


Figure 4: Mean $|\text{SHAP value}|$ for the rolling-CV-selected LightGBM (as of 2026-06-06). $\text{PM}_{2.5}$ and its log transform dominate; `doy_cos` captures the seasonal cycle; AOD and dust lead features reflect aerosol loading at forecast horizons; 14-day AQI rolling statistics encode medium-term pollution memory.

4 Training Pipeline & Models

`train.py` follows a two-step protocol on every daily run:

1. **Holdout evaluation** — time-aware split (last 90 days = test). Each model is trained and scored per horizon (d1–d4).
2. **Full retrain** — each model refit on all labeled rows (2023-01-01 to present, ~1 247 rows) and saved to `model_registry`; old versions are purged automatically.

A small multiplicative jitter ($\pm 5\%$) is applied to lead features at training time to bridge the train/serve gap caused by forecast error in production leads (deterministic seed 42).

PerHorizonWrapper (LGBM only). LightGBM trains one `LGBMRegressor` per forecast horizon, wrapped in `PerHorizonWrapper` so each sub-model fits its horizon-specific error distribution independently. Ridge uses scikit-learn’s `MultiOutputRegressor` and LSTM uses a native `Dense(4)` output layer — both produce all four horizons in a single forward pass.

Ridge Regression. `StandardScaler` + Ridge with $\alpha = 1.0$. Used as a fast linear baseline.

LightGBM. Per-horizon `LGBMRegressor` with L1/L2 regularization and a two-stage early-stopping protocol: (1) discover the optimal iteration on a 15% validation tail, (2) refit on all rows with that count scaled proportionally. This ensures the deployed model uses every observation without risking overfitting.

LSTM. Sequence-to-4 architecture, 7-day input window, two stacked LSTM layers (64 then 32 units, L_2 regularisation), dropout 0.4, `Dense(4)` output. Independent scalers for inputs and targets; trained for up to 150 epochs with early stopping (patience 25) and adaptive learning-rate reduction.

5 Key Blockers and How They Were Fixed

5.1 LightGBM Severe Overfitting ($10.97\times$ train/test RMSE ratio)

Problem. The original configuration used `n_estimators=500`, `max_depth=7`, `num_leaves=63`, no regularization, and a dead `subsample` parameter — `subsample_freq` defaulted to 0, so bagging never applied. The result was train $R^2 \approx 0.999$ and test/train RMSE ratio of $10.97\times$.

Fix. `lgbm_model.py` was rewritten with the following changes:

Table 2: LightGBM parameter changes

Parameter	Before	After
<code>n_estimators</code>	500 (fixed)	2000 (ceiling, early-stop)
<code>max_depth</code>	7	4
<code>num_leaves</code>	63	15
<code>subsample_freq</code>	0 (dead)	1 (active)
<code>min_child_samples</code>	default	30
<code>reg_alpha</code>	0	0.1
<code>reg_lambda</code>	0	1.0
Early stopping	none	50 rounds on val tail

Result: train/test RMSE ratio dropped from $10.97\times$ to $1.40\times$. LightGBM was subsequently confirmed by rolling-origin CV as the best model across all horizons.

5.2 Misleading 60-Day Single Holdout

Problem. A narrow 60-day holdout (the most recent, calm-weather window) showed Ridge winning (RMSE 10.19) with LightGBM a close second. This was a window-selection artifact: the calm recent tail flattered linear and autoregressive models.

Diagnosis. Multi-window holdouts (60, 90, 120, 180, 270 days) and rolling-origin cross-validation (5-fold TimeSeriesSplit + Diebold-Mariano test) were run. Results completely reversed the 60-day conclusion:

Table 3: Test RMSE by holdout window (lower = better)

Holdout days	Ridge	LGBM	LSTM
60	10.19	10.54	11.64
90	12.73	11.88	11.10
180	14.67	13.70	19.73
270	16.14	13.71	17.65
Rolling-origin CV	28.03 $\pm$ 22.34	16.69$\pm$4.22	22.96 $\pm$ 9.69

Fix. The 60-day result was a window-selection artifact. Broader evaluation windows (90, 180, 270 days) and rolling-origin CV consistently reversed this ranking, with LightGBM leading across most windows; it has consistently been selected by rolling-origin CV in recent runs.

5.3 PM_{2.5} Lead Features and the Shadow Model

Observation. Including PM_{2.5} forecasts ($t+1$ to $t+4$) as input features yields $R^2 \approx 0.91$ — unsurprising, since PM_{2.5} is the dominant AQI driver ($r = 0.91$) and the model effectively depends on it almost entirely.

Why they are excluded from production. The Open-Meteo air quality API supplying these values is unstable: the same future date can return materially different PM_{2.5} readings depending on when the API is queried. The offline metrics therefore cannot be reliably reproduced in production,

so $\text{PM}_{2.5}$ leads are excluded (`EXCLUDE_COLS` in `train.py`).

Shadow model. A separate *shadow forecast* (`scripts/predict_pm25_shadow.py`) trains the same model family *with* $\text{PM}_{2.5}$ leads and writes results to a separate `predictions_pm25` collection for ongoing research — without affecting the live dashboard.

5.4 NNLS Ensemble: Added, Then Retired

Context. Before LightGBM overfitting was fixed, the $10.97\times$ train/test RMSE ratio made LightGBM unreliable as a standalone model. A non-negative least squares (NNLS) ensemble blend of all three models was introduced to compensate: per-horizon weights were fit on a held-out window, letting Ridge and LSTM down-weight the overfit LightGBM automatically. This improved overall accuracy and was used in production.

Problem after the fix. Once LightGBM was regularised (ratio $1.40\times$), the ensemble became a liability. On the most recent calm-weather window the blend over-trusted LSTM at d2–d4, widening the mean error against Open-Meteo from ≈ 2.4 AQI (LightGBM-only) to ≈ 5.9 AQI (blend). The ensemble weights were fit on a historical holdout that did not represent the current regime, so the blend was systematically mis-weighted at serving time.

Fix. The ensemble was retired. A lightweight rolling-origin CV (3-fold `TimeSeriesSplit` on the full dataset) now selects a single best model to serve each run, falling back to LightGBM on any failure. This is implemented in `_rolling_cv_best()` in `train.py`.

5.5 Open-Meteo API Reliability

Problem. GitHub Actions runners intermittently experienced timeouts and 5xx errors from Open-Meteo, causing entire pipeline runs to fail.

Fix. Retry logic with exponential back-off (up to 3 attempts; 1 s and 2 s waits) was added to the preprocessing module. API timeout raised to 60 s and max retries to 5 for the prediction step. If the forecast-fill API fails entirely, NaN lead features are imputed at inference time using column medians from historical rows.

5.6 PKT Timezone Gap (00:00–05:00)

Problem. Hourly data fetched for “today” in UTC returned no rows for midnight–5 AM PKT because those hours fall on the next UTC calendar day, leaving a 5-hour gap in the daily aggregate.

Fix. The fetch end-date was switched from UTC to the PKT calendar date, ensuring the daily average always covers a full 24 hours.

6 Results

6.1 Production Model ($\text{PM}_{2.5}$ Leads Excluded)

Table 4: 90-day holdout metrics — production configuration (141 features). † Rolling-origin CV selection (3 folds, full dataset).

Model	MAE	RMSE	R^2
Ridge	9.36	12.68	0.500
LSTM	9.48	11.95	0.557
LightGBM [†]	8.77	11.96	0.555

Table 5: LightGBM per-horizon metrics (rolling-CV-selected model, 90-day holdout)

Horizon	MAE	RMSE	R ²
d1	5.57	7.51	0.825
d2	9.15	12.07	0.547
d3	10.04	13.30	0.448
d4	10.34	13.91	0.400

6.2 PM_{2.5} Shadow Model (Research Only)

The shadow pipeline trains the same model family *with* PM_{2.5} leads included (145 features). Given how completely AQI depends on PM_{2.5}, the metrics are strong but are driven by the API-supplied leads discussed in Section 5.3; the shadow model is not deployed.

Table 6: 90-day holdout metrics — PM_{2.5} shadow configuration (145 features, with PM_{2.5} leads)

Model	MAE	RMSE	R ²
Ridge	4.55	5.93	0.891
LSTM	4.27	5.93	0.891
LightGBM	3.97	5.51	0.906

Table 7: LightGBM per-horizon metrics — PM_{2.5} shadow

Horizon	MAE	R ²
d1	3.7	0.92
d2	4.2	0.89
d3	4.0	0.91
d4	3.9	0.91

The shadow model’s RMSE of 5.51 vs. the production model’s 11.96 shows how completely the prediction depends on PM_{2.5}: excluding the unstable API-supplied leads costs ~6.5 RMSE units.

6.3 Live Sanity Check vs. Open-Meteo

Table 8: Our predictions vs. Open-Meteo `us_aqi` at all horizons (all dates 2026). *dn* = forecast made *n* days before the target date. OM: reanalysis actuals (past); live forecast snapshot (future, values may vary between fetches). Dashes indicate a prediction was not yet available on that anchor day.

Date	OM	d1	d2	d3	d4
<i>Past (vs. reanalysis actuals)</i>					
05-27	68.1	71.0	—	—	—
05-28	70.9	70.7	69.6	—	—
05-29	73.7	69.3	67.8	69.1	—
05-30	71.6	70.2	70.9	66.0	68.7
05-31	68.2	65.3	68.9	71.1	66.2
06-01	70.8	68.0	67.5	73.6	70.2
<i>Current forecast (anchor 06-04, vs. OM live forecast)</i>					
06-05	92.5	90.6	—	—	—
06-06	79.9	—	82.6	—	—
06-07	72.0	—	—	77.9	—
06-08	74.6	—	—	—	78.6

7 Automated CI/CD Pipeline

Two GitHub Actions workflows, both triggered by `workflow_dispatch` from an external cron-job.org timer (no GitHub-native cron required, avoiding cold-start delays):

Feature Pipeline (every hour): fetch new daily data → fetch new hourly data → run feature engineering.

Training Pipeline (daily 7:19 AM PKT, scheduled to allow Open-Meteo values to stabilise and provide a better anchor): train all models → generate forecast for the current day and the three days following → generate $PM_{2.5}$ shadow forecast → generate SHAP explanations.

MongoDB credentials are stored as GitHub Secrets. Every run starts cold from the repository; no local state persists between runs.

8 Dashboard & Explainability

The Streamlit dashboard reads from MongoDB at startup and displays:

- Next-4-day AQI forecast tiles, colour-coded to the US AQI scale
- Intraday hourly AQI trend chart
- Historical feature-store trend and EDA charts (including the monthly boxplot)
- AQI alert banner for any forecasted day > 150 (Unhealthy)

9 Limitations & Future Work

- **Data volume.** `load_data()` filters to 2023-01-01, using only ~1247 of the ~3077 rows in the feature store (AQ labels begin Aug 2022; records before 2023 are currently unused). Learning curves show LGBM validation RMSE still descending — extending the window is the highest-return improvement available. LSTM, which showed strong architectural promise, was largely unable to realise that potential on this dataset size.
- **Ridge regularisation.** $\alpha = 1.0$ is under-regularised for 141 features; RidgeCV would be a straightforward improvement.
- **$PM_{2.5}$ shadow.** If the Open-Meteo $PM_{2.5}$ forecast values can be shown to be sufficiently stable and accurate, including them in production could reduce MAE by up to 4.7 AQI units.

10 Conclusion

Breathe Karachi delivers a production-grade, fully automated AQI forecasting service for Karachi using only free-tier cloud infrastructure. The key engineering lessons were: (1) a narrow holdout window gives dangerously wrong model rankings — rolling-origin CV is essential for time-series evaluation; (2) LightGBM’s bagging was silently disabled by a default parameter, causing $10.97\times$ overfitting that was fixed with a regularised parameter set and early stopping; and (3) $PM_{2.5}$ leads inflate offline metrics by ~6.4 RMSE units, but the Open-Meteo air quality API supplying them is unstable — values vary between fetches — so the gain is not reliably realizable in production. The rolling-origin-CV-selected model (LightGBM in the current run) achieves 3.6 AQI mean error against Open-Meteo forecast values — a practically useful forecast for daily air quality planning.